1. Preprocessing

1.1 Load library

In [2]:
```
import os
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image

array = np.ndarray

from itertools import product
import pandas as pd
```

1.2 Load ORL Dataset and Extended YaleB Dataset.

- ORL dataset contains ten different images of each of 40 distinct subjects. For some subjects, the images were taken at different times, varying the lighting, facial expressions (open / closed eyes, smiling / not smiling) and facial details (glasses / no glasses). All the images were taken against a dark homogeneous background with the subjects in an upright, frontal position (with tolerance for some side movement). The size of each image is 92x112 pixels, with 256 grey levels per pixel. To further reduce the computation complexity, you can resize all images to 30x37 pixels.

- Extended YaleB dataset contains 2414 images of 38 human subjects under 9 poses and 64 illumination conditions. All images are manually aligned, cropped, and then resized to 168x192 pixels. To further reduce the computation complexity, you can resize all images to 42x48 pixels.

In [3]:
```
root_ORL = 'data/ORL'
img_resize_ORL = (37,30)
root_YaleB = 'data/CroppedYaleB'
img_resize_YaleB = (48,42)
max_iter_nmf = 5000
max_iter_L21_nmf = 5000
```

In [4]:
```
import logging

# Create logger
logger = logging.getLogger("logger")
logger.setLevel(logging.DEBUG)

train_handler = logging.FileHandler("train.log", mode="w", encoding="utf-
train_handler.setLevel(logging.INFO)
train_formatter = logging.Formatter("%(asctime)s - %(levelname)s - %(mess
train_handler.setFormatter(train_formatter)
```

```
debug_handler = logging.FileHandler("debug.log", mode="w", encoding="utf-
debug_handler.setLevel(logging.DEBUG)
debug_formatter = logging.Formatter(
    "%(asctime)s - [%(levelname)s] - %(filename)s:%(lineno)d - %(message)
)
debug_handler.setFormatter(debug_formatter)

logger.addHandler(train_handler)
logger.addHandler(debug_handler)

result_logger = logging.getLogger("result_logger")
result_logger.setLevel(logging.INFO)

result_handler = logging.FileHandler("result.log", mode="w", encoding="ut
result_formatter = logging.Formatter("%(asctime)s - [RESULT] - %(message)
result_handler.setFormatter(result_formatter)

result_logger.addHandler(result_handler)
result_logger.propagate = False
```

In [5]:

```
def load_data(root='data/CroppedYaleB', img_resize = (1,1)):
    """
    Load ORL (or Extended YaleB) dataset to numpy array.

    Args:
        root: path to dataset.
        reduce: scale factor for zooming out images.

    """
    images, labels = [], []

    for i, person in enumerate(sorted(os.listdir(root))):

        if not os.path.isdir(os.path.join(root, person)):
            continue

        for fname in os.listdir(os.path.join(root, person)):

            # Remove background images in Extended YaleB dataset.
            if fname.endswith('Ambient.pgm'):
                continue

            if not fname.endswith('.pgm'):
                continue

            # load image.
            img = Image.open(os.path.join(root, person, fname))
            img = img.convert('L') # grey image.

            # reduce computation complexity.
            img = img.resize(img_resize)

            # TODO: preprocessing.

            # convert image to numpy array.
            img = np.asarray(img).reshape((-1,1)).astype(np.float32)
            img = img / 255.0  # normalization to [0,1]
```

```
            # collect data and label.
            images.append(img)
            labels.append(i)

    # concate all images and labels.
    images = np.concatenate(images, axis=1)
    labels = np.array(labels)

    return images, labels
```

In [6]:
```
# Load ORL dataset.
ORL_X, ORL_Y = load_data(root=root_ORL, img_resize=img_resize_ORL)
print('ORL dataset: X.shape = {}, Y.shape = {}'.format(ORL_X.shape, ORL_Y

# Load Extended YaleB dataset.
YaleB_X, YaleB_Y = load_data(root=root_YaleB, img_resize = img_resize_Yal
print('Extended YalB dataset: X.shape = {}, Y.shape = {}'.format(YaleB_X.
```

```
ORL dataset: X.shape = (1110, 400), Y.shape = (400,)
Extended YalB dataset: X.shape = (2016, 2414), Y.shape = (2414,)
```

1.2 Dataset pre-show

In [7]:
```
# A function to show the examples in dataset.

def examples(dataset, n_rows = 5, n_cols = 10, img_resize = (1,1,1)):

    faces = dataset.T.reshape(img_resize)

    plt.figure(figsize=(n_cols, n_rows))
    plt.axis('off')

    for r in range(n_rows):
        for c in range(n_cols):

            index = r * 10 + c

            plt.subplot(n_rows, n_cols, index+1)
            plt.axis('off')
            plt.imshow(faces[index], cmap= "gray" )


    plt.show()
```

In [8]:
```
print(f"Some examples img of ORL dataset")
examples(ORL_X, img_resize = (400,30,37))
print(f"First 10 examples of y_train: {ORL_Y[0:10]}")
```

```
Some examples img of ORL dataset
```

```
First 10 examples of y_train: [2 2 2 2 2 2 2 2 2 2]
```

In [9]:
```
print(f"Some examples img of YaleB dataset")
examples(YaleB_X, img_resize = (2414,42,48))
print(f"First 10 examples of y_train: {YaleB_Y[0:10]}")
```

```
Some examples img of YaleB dataset
```

```
First 10 examples of y_train: [1 1 1 1 1 1 1 1 1 1]
```

2. Main NMF Algorithm

2.1 Standard NMF (L2 NMF) Algorithm

In [10]:
```
def nmf_standard(V, y, eps=1e-10, seed=0, maxiter = max_iter_nmf):
    """
    Standard NMF (L2 NMF)
    V: d * n non-negative matrix
    k: rank for factorization
    """
```

```
    d, n = V.shape

    # Initial iter matrix
    # Here rank is usually to choose close to the examples of dataset.
    k = len(np.unique(y))

    # init W, H
    np.random.seed(seed)
    W = np.random.random((d, k))
    H = np.random.random((k, n))

    diff = float('Inf')
    iters = 0

    while (iters < maxiter) and (diff > eps):
        # update H
        WH = np.dot(W, H)
        H_new = H * (np.dot(W.T, V)) / (np.dot(W.T, WH) + 1e-12)

        # update W
        WH = np.dot(W, H_new)
        W_new = W * (np.dot(V, H_new.T)) / (np.dot(W, np.dot(H_new, H_new

        # non-negativity
        W_new = np.clip(W_new, 0, None)
        H_new = np.clip(H_new, 0, None)

        # check convergence
        diff_W = np.linalg.norm(W_new - W)
        diff_H = np.linalg.norm(H_new - H)
        diff = diff_W + diff_H

        # update W, H
        W, H = W_new, H_new

        iters += 1

    logger.debug(f"Converged after {iters} iterations, diff={diff:.3e}")
    return W, H
```

2.2 L2,1-Norm Based NMF Algorithm

In [11]:
```
# Function to check if the Matrix is positive or not.

def check_positive(X:array):
    if np.any(X<0):
        raise ValueError(f'The matrix {X} is not a Non-negative Matrix')
```

In [12]:
```
# Algorithm

def L21_norm_based_NMF(X:array, y:array, name:str,
                                random_state:int | None = 42,
                                max_iter = max_iter_L21_nmf,
                                convergence_criterion = 1e-7):
    '''
```

```
    '''
    check_positive(X)
    m,n = X.shape
    rng = np.random.default_rng(random_state)

    # In case to happen that the Denominator=0
    eps = 1e-12

    # Record the iteration
    history = []

    # Initial iter matrix
    # Here rank is usually to choose close to the examples of dataset.
    rank = len(np.unique(y))

    F = rng.random((m,rank))+1e-8
    G = rng.random((rank,n))+1e-8

    X_sq = np.sum(X * X, axis=0)


    for i in range(max_iter):

        # Compute D Matrix
        F_G = F @ G
        F_G_sq = np.sum(F_G * F_G, axis=0)
        cross = np.sum(X * F_G, axis=0)
        D_denominator = np.sqrt(np.maximum(X_sq + F_G_sq - 2 * cross,0.0)
        D = 1.0 / np.maximum(D_denominator,eps)

        # Update F
        X_D = X * D
        G_D = G * D
        F_molecular = X_D @ G.T
        F_denominator = F @ (G_D @ G.T) + eps
        F_denominator = np.maximum(F_denominator,eps)

        np.divide(F_molecular, F_denominator, out=F_molecular)
        F *= F_molecular
        F = np.maximum(F,eps) # Make sure non-negative.

        # Update G
        G_molecular = F.T @ X_D
        G_denominator = F.T @ F @ G * D + eps
        G_denominator = np.maximum(G_denominator,eps)

        np.divide(G_molecular, G_denominator, out=G_molecular)
        G *= G_molecular
        G = np.maximum(G,eps) # Make sure non-negative.

        # Update residual
        residual = D_denominator
        history.append(np.sum(residual))

        if i > 0:
            convergence_new = abs(history[-1] - history[-2]) / (history[-
```

```
            if convergence_new < convergence_criterion:
                logger.debug(f'Reached the convergence criterion. Total i
                break

        if i == max_iter - 1:
            logger.debug(f'Reached the max iteration. Last convergence ra

    return F, G
```

3. Evaluation Metrics

3.1 Add salt & pepper noise

Pre-show of noise plot.

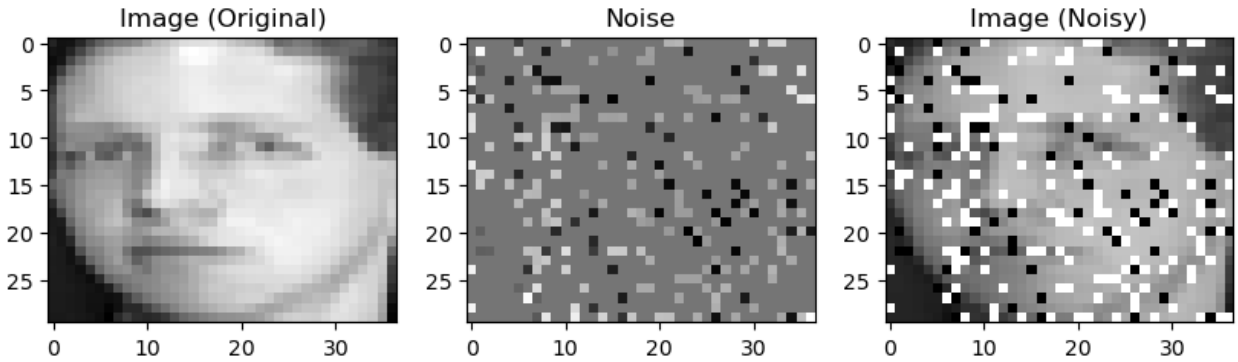
In [13]:
```
# Add salt & pepper noise Algorithm

def add_salt_pepper(X, p, r, seed, vmax=1.0):
    """
    Args:
        X    : numpy array of shape (d, n), values in [0,1].
        p    : fraction of pixels to be corrupted per image.
        r    : fraction of corrupted pixels set to salt (vmax); the rest
        seed : random seed (to be reproducible).
        vmax : maximum pixel value (default 1.0 after normalization).
    """
    d, n = X.shape
    np.random.seed(seed)
    Xn = X.copy()

    m = int(round(p * d))     # number of pixels to corrupt per column
    if m <= 0:
        return Xn
    k = int(round(r * m))     # number of salt pixels (set to vmax)

    for j in range(n):
        idx = np.random.choice(d, size=m, replace=False)
        # salt: set selected pixels to vmax
        Xn[idx[:k], j] = vmax
        # pepper: set the remaining to 0
        Xn[idx[k:], j] = 0.0
    return Xn
```

In [14]:
```
# Plot pre-show

def Noise_pre_show(root, img_resize, name:str):

    # Load dataset.
    print(f'==> Load {name} dataset ...')
    V_hat, Y_hat = load_data(root=root, img_resize = img_resize)
    print('V_hat.shape={}, Y_hat.shape={}'.format(V_hat.shape, Y_hat.shap
```

```
    # Add salt & pepper noise.
    p, r = 0.2, 0.7
    V = add_salt_pepper(V_hat, p=p, r=r, seed=0, vmax=1.0)

    # Derive a pure noise image (for visualization).
    V_noise = V - V_hat

    # Plot result.
    img_size = img_resize
    ind = 2  # index of demo image.

    plt.figure(figsize=(10,3))

    # Original
    plt.subplot(131)
    plt.imshow(V_hat[:,ind].reshape(img_size[1], img_size[0]), cmap=plt.c
    plt.title('Image (Original)')

    # Noise only
    plt.subplot(132)
    plt.imshow(V_noise[:,ind].reshape(img_size[1], img_size[0]), cmap=plt
    plt.title('Noise')

    # Noisy image
    plt.subplot(133)
    plt.imshow(V[:,ind].reshape(img_size[1], img_size[0]), cmap=plt.cm.gr
    plt.title('Image (Noisy)')
    plt.show()
```

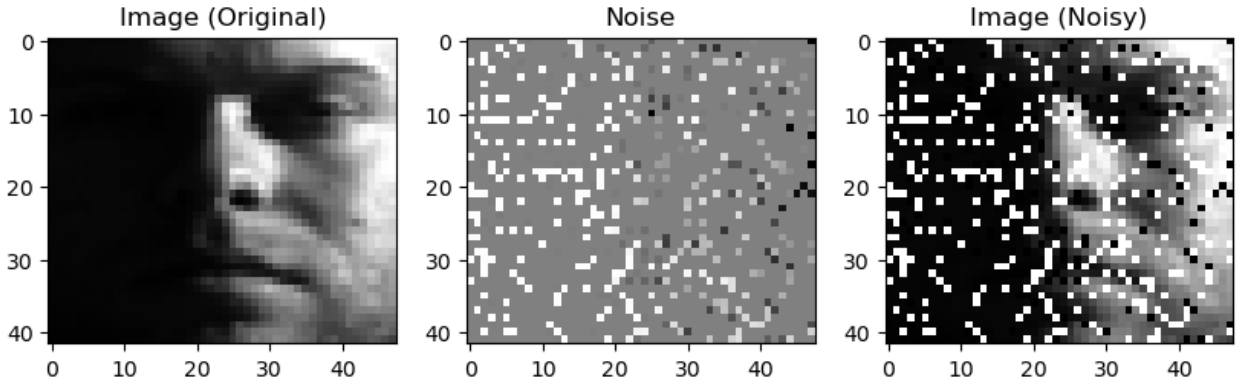
In [15]:
```
Noise_pre_show(root = root_ORL, img_resize=img_resize_ORL, name = "ORL")
```

```
==> Load ORL dataset ...
V_hat.shape=(1110, 400), Y_hat.shape=(400,)
```



In [16]:
```
Noise_pre_show(root = root_YaleB, img_resize=img_resize_YaleB, name = "Ya
```

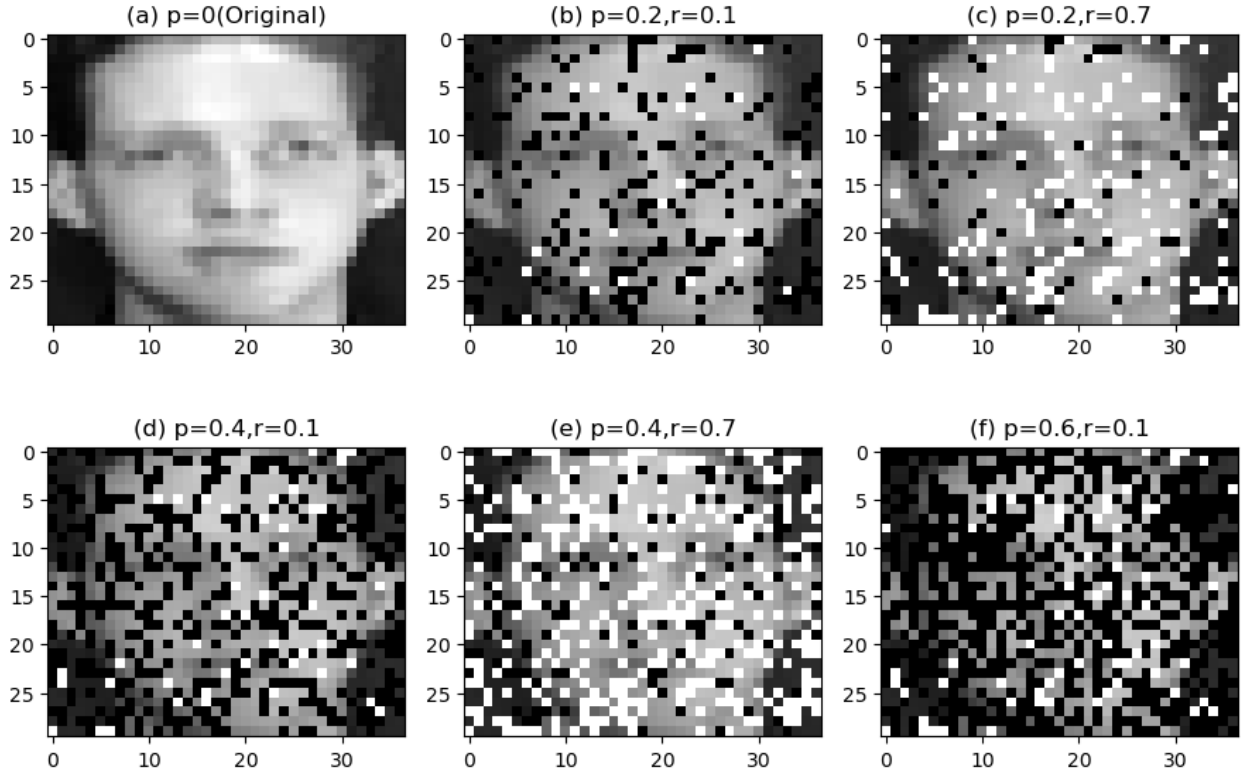
```
==> Load YaleB dataset ...
V_hat.shape=(2016, 2414), Y_hat.shape=(2414,)
```



In [17]:
```
# Plot pre-show below different p and r.

def Noise_pre_show_pr(root, img_resize, name:str):

    print(f'==> Load {name} dataset ...')
    V_hat, Y_hat = load_data(root=root, img_resize = img_resize)
    print('V_hat.shape={}, Y_hat.shape={}'.format(V_hat.shape, Y_hat.shap

    # Example index
    ind = 7
    img_size = img_resize

    # Original
    img_a = V_hat[:, ind].reshape(img_size[1], img_size[0])
    # p=0.2, r=0.1
    img_b = add_salt_pepper(V_hat, p=0.2, r=0.1, seed=0, vmax=1.0)[:, ind
    # p=0.2, r=0.7
    img_c = add_salt_pepper(V_hat, p=0.2, r=0.7, seed=0, vmax=1.0)[:, ind
    # p=0.4, r=0.1
    img_d = add_salt_pepper(V_hat, p=0.4, r=0.1, seed=0, vmax=1.0)[:, ind
    # p=0.4, r=0.7
    img_e = add_salt_pepper(V_hat, p=0.4, r=0.7, seed=0, vmax=1.0)[:, ind
    # p=0.6, r=0.1
    img_f = add_salt_pepper(V_hat, p=0.6, r=0.1, seed=0, vmax=1.0)[:, ind

    plt.figure(figsize=(9, 6))
    plt.subplot(2,3,1); plt.imshow(img_a, cmap='gray'); plt.title("(a) p=
    plt.subplot(2,3,2); plt.imshow(img_b, cmap='gray'); plt.title("(b) p=
    plt.subplot(2,3,3); plt.imshow(img_c, cmap='gray'); plt.title("(c) p=
    plt.subplot(2,3,4); plt.imshow(img_d, cmap='gray'); plt.title("(d) p=
    plt.subplot(2,3,5); plt.imshow(img_e, cmap='gray'); plt.title("(e) p=
    plt.subplot(2,3,6); plt.imshow(img_f, cmap='gray'); plt.title("(f) p=
    plt.tight_layout()
    plt.show()
```

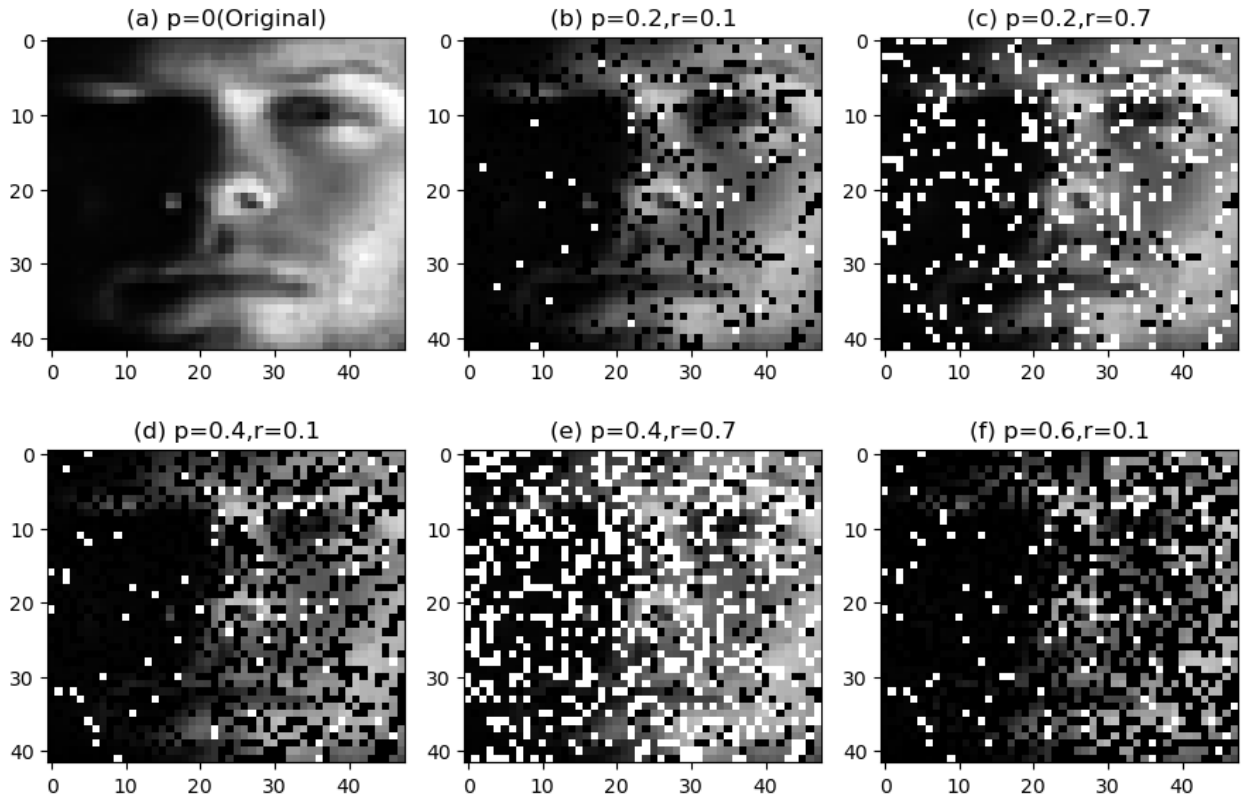
In [18]:
```
Noise_pre_show_pr(root = root_ORL, img_resize=img_resize_ORL, name = "ORL
```

```
==> Load ORL dataset ...
V_hat.shape=(1110, 400), Y_hat.shape=(400,)
```



In [19]:
```
Noise_pre_show_pr(root = root_YaleB, img_resize=img_resize_YaleB, name =
```

```
==> Load YaleB dataset ...
V_hat.shape=(2016, 2414), Y_hat.shape=(2414,)
```



3.2 Calculate W and H for each parameters and iterations

In [20]:
```
WH_records = {}

# Function to record W & H once
```

```python
def record_WH(p, r, method:str, name:str, ratio = 0.9, num=5, WH_records
    """
    Origin_X : clean data matrix (d * n)
    p, r    : noise parameters
    k       : rank for NMF
    num     : number of repetitions
    seed    : random seed
    add_noise: function to add noise
    nmf_func : NMF function
    L_21_norm_based_NMF : L_21_norm_based_NMF function
    """
    if name == "ORL":
        Origin_X,Origin_y = load_data(root = root_ORL, img_resize = img_r
    if name == "YaleB":
        Origin_X,Origin_y = load_data(root = root_YaleB, img_resize = img

    n_samples = Origin_X.shape[1]
    n_train = int(n_samples * ratio)

    for t in range(num):

        logger.info(f'Start calculate W and H in ({name}) using ({method}

        # 90% sample
        rng = np.random.default_rng(seed=t)
        indices = rng.permutation(n_samples)

        # Collect the ratio of data.
        test_idx = indices[:n_train]
        X_test, y_test = Origin_X[:,test_idx], Origin_y[test_idx]
        V_hat, Y_hat = X_test, y_test

        # add salt & pepper noise
        V_noisy = add_salt_pepper(V_hat, p=p, r=r, seed=seed+t, vmax=1.0)

        # factorize noisy data
        if method == "L2_norm_NMF":
            W, H = nmf_standard(V_noisy, Y_hat, eps=1e-7)
        if method == "L21_norm_based_NMF":
            W, H = L21_norm_based_NMF(X = V_noisy, y = Y_hat, name = name

        WH_records[(name, method, p, r, t, "W")] = np.asarray(W)
        WH_records[(name, method, p, r, t, "H")] = np.asarray(H)
        WH_records[(name, method, p, r, t, "V_hat")] = np.asarray(V_hat)
        WH_records[(name, method, p, r, t, "Y_hat")] = np.asarray(Y_hat)

        logger.info(f'W = {W}')
        logger.info(f'H = {H}')
        logger.info(f'--------------------------------------------')

    return WH_records

# Function to evaluate all dataset and algorithm

def All_WH():
    datasets = ["ORL", "YaleB"]
    methods = ["L2_norm_NMF","L21_norm_based_NMF"] #
```

```
    ps = [0.2,0.4,0.6]
    rs = [0.1,0.4,0.7]

    for p, r, name, method in product(ps, rs, datasets, methods):
        WH_records = record_WH(p = p, r = r, method = method, name = name

    return WH_records
```

In [21]:
```
WH_records = All_WH()
```

3.3 Relative Reconstruction Errors (RRE)

To compare the robustness of different NMF algorithms, you can use the `relative reconstruction errors`. Let V denote the contaminated dataset (by adding noise), and $\hat{V}$ denote the clean dataset. Let W and H denote the factorization results on V, the `relative reconstruction errors` then can be defined as follows:

$$RRE = \frac{\|\hat{V} - WH\|_F}{\|\hat{V}\|_F}. \tag{1}$$

In [22]:
```
# Main function
def rre_score(W,H,V_hat):
    rre = np.linalg.norm(V_hat - W.dot(H),'fro') / np.linalg.norm(V_hat,'
    return rre

def calculate_rre(WH_records):

    datasets = ["ORL", "YaleB"]
    methods = ["L2_norm_NMF","L21_norm_based_NMF"]
    ps = [0.2,0.4,0.6]
    rs = [0.1,0.4,0.7]
    seed = range(5)

    rre_records = {}

    for p, r, name, method in product(ps, rs, datasets, methods):
        rre_list = []
        for t in seed:
            W = WH_records[name, method, p, r, t, "W"]
            H = WH_records[name, method, p, r, t, "H"]
            V_hat = WH_records[name, method, p, r, t, "V_hat"]
            rre_list.append(rre_score(W,H,V_hat))

        mean = round(float(np.mean(rre_list)),8)
        std = round(float(np.std(rre_list)),8)

        logger.info(f'RRE score on ({name}) using ({method}) with p = {p}

        rre_records[(name, method, float(p), float(r))] = {
        "rre_mean": mean,
        "rre_std": std,
        }
```

```
    return rre_records
```

In [23]:
```
rre_records = calculate_rre(WH_records)
```

In [24]:
```
# Summary rre records

rows = [
    {
        "dataset": k[0],
        "nmf_method": k[1],
        "p": k[2],
        "r": k[3],
        "rre_mean": v["rre_mean"],
        "rre_std": v["rre_std"],
    }
    for k, v in rre_records.items()
]

df_rre = pd.DataFrame(rows)
print(df_rre)
result_logger.info(f'RRE Results:')
result_logger.info(f'{df_rre}')
```

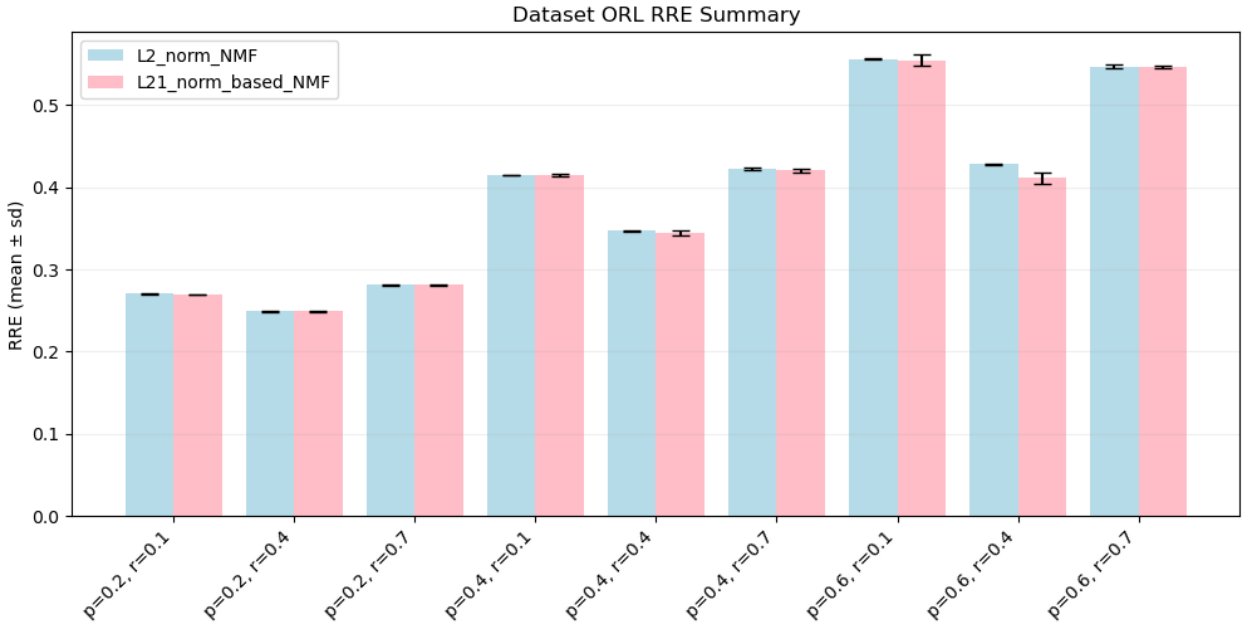
```
   dataset          nmf_method    p    r  rre_mean   rre_std
0      ORL         L2_norm_NMF  0.2  0.1  0.270295  0.000517
1      ORL  L21_norm_based_NMF  0.2  0.1  0.269087  0.000582
2    YaleB         L2_norm_NMF  0.2  0.1  0.257389  0.000547
3    YaleB  L21_norm_based_NMF  0.2  0.1  0.256240  0.000320
4      ORL         L2_norm_NMF  0.2  0.4  0.248753  0.000753
5      ORL  L21_norm_based_NMF  0.2  0.4  0.248951  0.000938
6    YaleB         L2_norm_NMF  0.2  0.4  0.261670  0.000199
7    YaleB  L21_norm_based_NMF  0.2  0.4  0.261725  0.000645
8      ORL         L2_norm_NMF  0.2  0.7  0.281011  0.000697
9      ORL  L21_norm_based_NMF  0.2  0.7  0.280894  0.000928
10   YaleB         L2_norm_NMF  0.2  0.7  0.355032  0.000568
11   YaleB  L21_norm_based_NMF  0.2  0.7  0.354162  0.000653
12     ORL         L2_norm_NMF  0.4  0.1  0.414936  0.000245
13     ORL  L21_norm_based_NMF  0.4  0.1  0.414607  0.000931
14   YaleB         L2_norm_NMF  0.4  0.1  0.394050  0.000305
15   YaleB  L21_norm_based_NMF  0.4  0.1  0.392321  0.000620
16     ORL         L2_norm_NMF  0.4  0.4  0.347132  0.000819
17     ORL  L21_norm_based_NMF  0.4  0.4  0.344625  0.002822
18   YaleB         L2_norm_NMF  0.4  0.4  0.405109  0.000638
19   YaleB  L21_norm_based_NMF  0.4  0.4  0.404892  0.000726
20     ORL         L2_norm_NMF  0.4  0.7  0.421825  0.001625
21     ORL  L21_norm_based_NMF  0.4  0.7  0.420183  0.002430
22   YaleB         L2_norm_NMF  0.4  0.7  0.616722  0.001437
23   YaleB  L21_norm_based_NMF  0.4  0.7  0.615216  0.001507
24     ORL         L2_norm_NMF  0.6  0.1  0.556032  0.000637
25     ORL  L21_norm_based_NMF  0.6  0.1  0.554477  0.006578
26   YaleB         L2_norm_NMF  0.6  0.1  0.545179  0.000218
27   YaleB  L21_norm_based_NMF  0.6  0.1  0.544295  0.000264
28     ORL         L2_norm_NMF  0.6  0.4  0.427719  0.001062
29     ORL  L21_norm_based_NMF  0.6  0.4  0.410944  0.007375
30   YaleB         L2_norm_NMF  0.6  0.4  0.551301  0.000974
31   YaleB  L21_norm_based_NMF  0.6  0.4  0.551347  0.001026
32     ORL         L2_norm_NMF  0.6  0.7  0.546857  0.001721
33     ORL  L21_norm_based_NMF  0.6  0.7  0.546262  0.001395
34   YaleB         L2_norm_NMF  0.6  0.7  0.875648  0.002119
35   YaleB  L21_norm_based_NMF  0.6  0.7  0.875193  0.002010
```

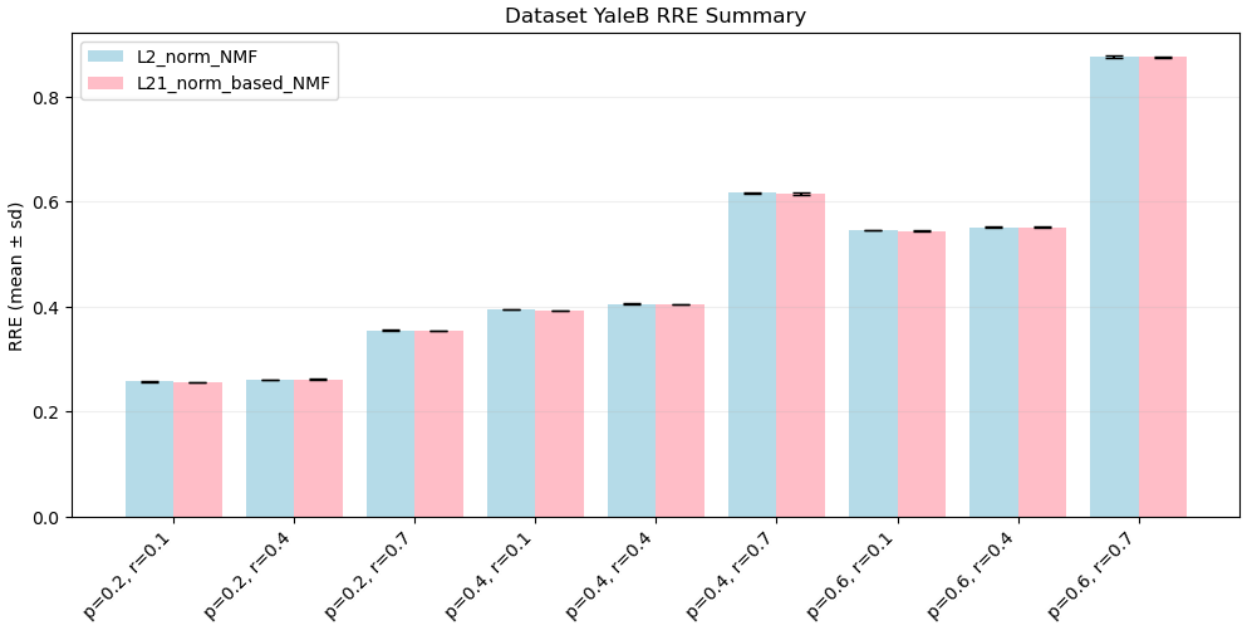
In [25]:

```
# Summary for the combination of p and r (RRE)
datasets = df_rre["dataset"].unique()

for ds in datasets:
    df_ds = df_rre[df_rre["dataset"] == ds].copy()
    pr_records = []
    prs = df_ds[["p","r"]].drop_duplicates().sort_values(["p","r"]).value

    for ps, rs in prs:
        sub_l2 = df_ds[(df_ds["nmf_method"]=="L2_norm_NMF") &
                       (df_ds["p"]==ps) & (df_ds["r"]==rs)]

        sub_l21 = df_ds[(df_ds["nmf_method"]=="L21_norm_based_NMF") &
                        (df_ds["p"]==ps) & (df_ds["r"]==rs)]

        pr_records.append([
            f"p={ps}, r={rs}",
            "L2",
            float(sub_l2["rre_mean"].values[0]),
```

```
            float(sub_l2["rre_std"].values[0]),
            "L21",
            float(sub_l21["rre_mean"].values[0]),
            float(sub_l21["rre_std"].values[0]),
        ])

    pr_summary = pd.DataFrame(pr_records, columns=[
        "pr", "L2", "L2_mean", "L2_std",
        "L21", "L21_mean", "L21_std"
    ])
    print(f"\nDataset {ds} RRE Summary")
    print(pr_summary)
    result_logger.info(f'Dataset {ds} RRE Summary')
    result_logger.info(f'{pr_summary}')

    #bar plot
    x = np.arange(len(pr_summary))
    w = 0.4

    fig, ax = plt.subplots(figsize=(12, 5))
    ax.bar(x - w/2, pr_summary["L2_mean"],  yerr=pr_summary["L2_std"],  w
    ax.bar(x + w/2, pr_summary["L21_mean"], yerr=pr_summary["L21_std"], w

    ax.set_xticks(x)
    ax.set_xticklabels(pr_summary["pr"], rotation=45, ha="right")
    ax.set_ylabel("RRE (mean ± sd)")
    ax.set_title(f"Dataset {ds} RRE Summary")
    ax.legend()
    ax.grid(axis="y", alpha=0.2)
```

```
Dataset ORL RRE Summary
             pr  L2   L2_mean    L2_std  L21  L21_mean   L21_std
0  p=0.2, r=0.1  L2  0.270295  0.000517  L21  0.269087  0.000582
1  p=0.2, r=0.4  L2  0.248753  0.000753  L21  0.248951  0.000938
2  p=0.2, r=0.7  L2  0.281011  0.000697  L21  0.280894  0.000928
3  p=0.4, r=0.1  L2  0.414936  0.000245  L21  0.414607  0.000931
4  p=0.4, r=0.4  L2  0.347132  0.000819  L21  0.344625  0.002822
5  p=0.4, r=0.7  L2  0.421825  0.001625  L21  0.420183  0.002430
6  p=0.6, r=0.1  L2  0.556032  0.000637  L21  0.554477  0.006578
7  p=0.6, r=0.4  L2  0.427719  0.001062  L21  0.410944  0.007375
8  p=0.6, r=0.7  L2  0.546857  0.001721  L21  0.546262  0.001395

Dataset YaleB RRE Summary
             pr  L2   L2_mean    L2_std  L21  L21_mean   L21_std
0  p=0.2, r=0.1  L2  0.257389  0.000547  L21  0.256240  0.000320
1  p=0.2, r=0.4  L2  0.261670  0.000199  L21  0.261725  0.000645
2  p=0.2, r=0.7  L2  0.355032  0.000568  L21  0.354162  0.000653
3  p=0.4, r=0.1  L2  0.394050  0.000305  L21  0.392321  0.000620
4  p=0.4, r=0.4  L2  0.405109  0.000638  L21  0.404892  0.000726
5  p=0.4, r=0.7  L2  0.616722  0.001437  L21  0.615216  0.001507
6  p=0.6, r=0.1  L2  0.545179  0.000218  L21  0.544295  0.000264
7  p=0.6, r=0.4  L2  0.551301  0.000974  L21  0.551347  0.001026
8  p=0.6, r=0.7  L2  0.875648  0.002119  L21  0.875193  0.002010
```





In [26]:

```
# Function to create table to visualize the result.
def table_visual(X:list,metric):
    table = X.pivot(index="p", columns="r", values=metric)
    print(table)
    result_logger.info(f'{table}')

# Table Visualization
metrics = ["rre_mean","rre_std"]
methods = ["L2_norm_NMF","L21_norm_based_NMF"]
ps = [0.2,0.4,0.6]
rs = [0.1,0.4,0.7]
print('The value of columns is r while the value of index is p.')

for metric, name, method in product(metrics, datasets, methods):
    print('-')
    print(f'RRE {metric} for dataset "{name}" using algorithm "{method}":
    result_logger.info(f'RRE {metric} for dataset "{name}" using algorith
    sub = df_rre[(df_rre["dataset"] == name) & (df_rre["nmf_method"] == m
    table_visual(sub, metric)
```

```
The value of columns is r while the value of index is p.
-
RRE rre_mean for dataset "ORL" using algorithm "L2_norm_NMF":
r         0.1       0.4       0.7
p
0.2  0.270295  0.248753  0.281011
0.4  0.414936  0.347132  0.421825
0.6  0.556032  0.427719  0.546857
-
RRE rre_mean for dataset "ORL" using algorithm "L21_norm_based_NMF":
r         0.1       0.4       0.7
p
0.2  0.269087  0.248951  0.280894
0.4  0.414607  0.344625  0.420183
0.6  0.554477  0.410944  0.546262
-
RRE rre_mean for dataset "YaleB" using algorithm "L2_norm_NMF":
r         0.1       0.4       0.7
p
0.2  0.257389  0.261670  0.355032
0.4  0.394050  0.405109  0.616722
0.6  0.545179  0.551301  0.875648
-
RRE rre_mean for dataset "YaleB" using algorithm "L21_norm_based_NMF":
r         0.1       0.4       0.7
p
0.2  0.256240  0.261725  0.354162
0.4  0.392321  0.404892  0.615216
0.6  0.544295  0.551347  0.875193
-
RRE rre_std for dataset "ORL" using algorithm "L2_norm_NMF":
r         0.1       0.4       0.7
p
0.2  0.000517  0.000753  0.000697
0.4  0.000245  0.000819  0.001625
0.6  0.000637  0.001062  0.001721
-
RRE rre_std for dataset "ORL" using algorithm "L21_norm_based_NMF":
r         0.1       0.4       0.7
p
0.2  0.000582  0.000938  0.000928
0.4  0.000931  0.002822  0.002430
0.6  0.006578  0.007375  0.001395
-
RRE rre_std for dataset "YaleB" using algorithm "L2_norm_NMF":
r         0.1       0.4       0.7
p
0.2  0.000547  0.000199  0.000568
0.4  0.000305  0.000638  0.001437
0.6  0.000218  0.000974  0.002119
-
RRE rre_std for dataset "YaleB" using algorithm "L21_norm_based_NMF":
r         0.1       0.4       0.7
p
0.2  0.000320  0.000645  0.000653
0.4  0.000620  0.000726  0.001507
0.6  0.000264  0.001026  0.002010
```

2.2 Evaluate Clustering Performance

1. Accuracy.

$$Acc(Y, Y_{pred}) = \frac{1}{n}\sum_{i=1}^{n} 1\{Y_{pred}(i) == Y(i)\}$$

2. Normalized Mutual Information (NMI).

$$NMI(Y, Y_{pred}) = \frac{2 * I(Y, Y_{pred})}{H(Y) + H(Y_{pred})}$$

where $I(\cdot, \cdot)$ is mutual information and $H(\cdot)$ is entropy.

In [27]:
```
from collections import Counter
from sklearn.cluster import KMeans

def assign_cluster_label(X, Y):
    kmeans = KMeans(n_clusters=len(set(Y))).fit(X)
    Y_pred = np.zeros(Y.shape)
    for i in set(kmeans.labels_):
        ind = kmeans.labels_ == i
        Y_pred[ind] = Counter(Y[ind]).most_common(1)[0][0] # assign label
    return Y_pred
```

In [28]:
```
from sklearn.metrics import accuracy_score
from sklearn.metrics import normalized_mutual_info_score

# Save Acc and NMI records
try:
    acc_nmi_records
except NameError:
    acc_nmi_records = []


# Evaluate Acc and NMI

def calculate_acc_nmi(WH_records):

    datasets = ["ORL", "YaleB"]
    methods = ["L2_norm_NMF","L21_norm_based_NMF"] #
    ps = [0.2,0.4,0.6]
    rs = [0.1,0.4,0.7]
    seed = range(5)

    acc_nmi_records = []

    for p, r, name, method in product(ps, rs, datasets, methods):
        acc_list = []
        nmi_list = []
        for t in seed:
            H = WH_records[name, method, p, r, t, "H"]
            Y_hat = WH_records[name, method, p, r, t, "Y_hat"]
```

```
            Y_pred = assign_cluster_label(H.T,Y_hat)
            acc = accuracy_score(Y_hat, Y_pred)
            acc_list.append(acc)
            nmi = normalized_mutual_info_score(Y_hat, Y_pred)
            nmi_list.append(nmi)

        acc_mean, acc_std = round(float(np.mean(acc_list)),8), round(floa
        nmi_mean, nmi_std = round(float(np.mean(nmi_list)),8), round(floa

        logger.info(f'ACC score on ({name}) using ({method}) with p = {p}
        logger.info(f'NMI score on ({name}) using ({method}) with p = {p}

        acc_nmi_records.append({
        "dataset": name,
        "nmf_method": method,
        "p": float(p),
        "r": float(r),
        "acc_mean": acc_mean,
        "acc_std": acc_std,
        "nmi_mean": nmi_mean,
        "nmi_std": nmi_std
        })

    return acc_nmi_records
```

In [29]:

```
# Summary for acc and NMI records
acc_nmi_records = calculate_acc_nmi(WH_records)
df_acc_nmi = pd.DataFrame(
    acc_nmi_records,
    columns=["dataset","nmf_method","p","r","acc_mean","acc_std","nmi_mea
).sort_values(["dataset","nmf_method","p","r"])

print(df_acc_nmi)
result_logger.info(f'\nResults for ACC and NMI')
result_logger.info(f'{df_acc_nmi}')
```

```
   dataset          nmf_method    p    r  acc_mean   acc_std  nmi_mean  \
1      ORL  L21_norm_based_NMF  0.2  0.1  0.533889  0.009196  0.694755
5      ORL  L21_norm_based_NMF  0.2  0.4  0.462778  0.028577  0.638062
9      ORL  L21_norm_based_NMF  0.2  0.7  0.440556  0.019452  0.620242
13     ORL  L21_norm_based_NMF  0.4  0.1  0.308333  0.017392  0.497793
17     ORL  L21_norm_based_NMF  0.4  0.4  0.248333  0.009230  0.437632
21     ORL  L21_norm_based_NMF  0.4  0.7  0.252222  0.012595  0.431450
25     ORL  L21_norm_based_NMF  0.6  0.1  0.208889  0.008678  0.366084
29     ORL  L21_norm_based_NMF  0.6  0.4  0.193333  0.008352  0.369256
33     ORL  L21_norm_based_NMF  0.6  0.7  0.200000  0.008784  0.376916
0      ORL         L2_norm_NMF  0.2  0.1  0.527778  0.039830  0.690813
4      ORL         L2_norm_NMF  0.2  0.4  0.480556  0.017830  0.643507
8      ORL         L2_norm_NMF  0.2  0.7  0.461667  0.018873  0.629385
12     ORL         L2_norm_NMF  0.4  0.1  0.299444  0.018291  0.492617
16     ORL         L2_norm_NMF  0.4  0.4  0.249444  0.011574  0.436613
20     ORL         L2_norm_NMF  0.4  0.7  0.259444  0.009230  0.441624
24     ORL         L2_norm_NMF  0.6  0.1  0.218889  0.009526  0.398566
28     ORL         L2_norm_NMF  0.6  0.4  0.195556  0.006236  0.383614
32     ORL         L2_norm_NMF  0.6  0.7  0.188333  0.002079  0.371728
3    YaleB  L21_norm_based_NMF  0.2  0.1  0.206538  0.014465  0.291029
```

```
7    YaleB  L21_norm_based_NMF  0.2  0.4  0.207735  0.006546  0.282401
11   YaleB  L21_norm_based_NMF  0.2  0.7  0.214917  0.005341  0.290274
15   YaleB  L21_norm_based_NMF  0.4  0.1  0.174954  0.008394  0.228220
19   YaleB  L21_norm_based_NMF  0.4  0.4  0.149448  0.003034  0.190000
23   YaleB  L21_norm_based_NMF  0.4  0.7  0.142541  0.001663  0.171786
27   YaleB  L21_norm_based_NMF  0.6  0.1  0.103591  0.003345  0.116367
31   YaleB  L21_norm_based_NMF  0.6  0.4  0.090700  0.003396  0.090926
35   YaleB  L21_norm_based_NMF  0.6  0.7  0.090055  0.003048  0.090465
2    YaleB         L2_norm_NMF  0.2  0.1  0.224217  0.011917  0.307366
6    YaleB         L2_norm_NMF  0.2  0.4  0.207643  0.011157  0.288455
10   YaleB         L2_norm_NMF  0.2  0.7  0.197145  0.010677  0.284925
14   YaleB         L2_norm_NMF  0.4  0.1  0.174770  0.012015  0.230837
18   YaleB         L2_norm_NMF  0.4  0.4  0.147422  0.004487  0.187951
22   YaleB         L2_norm_NMF  0.4  0.7  0.139411  0.009057  0.172081
26   YaleB         L2_norm_NMF  0.6  0.1  0.099171  0.004412  0.109132
30   YaleB         L2_norm_NMF  0.6  0.4  0.090424  0.002582  0.092143
34   YaleB         L2_norm_NMF  0.6  0.7  0.090331  0.001983  0.093060

     nmi_std
1   0.007602
5   0.028897
9   0.013712
13  0.012006
17  0.014093
21  0.020534
25  0.012294
29  0.007453
33  0.016814
0   0.030305
4   0.010367
8   0.014422
12  0.015867
16  0.017451
20  0.015249
24  0.009481
28  0.008029
32  0.012863
3   0.012011
7   0.006273
11  0.013864
15  0.005269
19  0.006190
23  0.012068
27  0.007354
31  0.002713
35  0.003612
2   0.012012
6   0.009477
10  0.009236
14  0.013544
18  0.008512
22  0.009829
26  0.006811
30  0.006386
34  0.003135
```

In [30]:
```
# Summary for the combination of p and r (Acc / NMI)
```

```python
datasets = df_acc_nmi["dataset"].unique()

for ds in datasets:
    df_ds = df_acc_nmi[df_acc_nmi["dataset"] == ds]
    prs = df_ds[["p","r"]].drop_duplicates().sort_values(["p","r"]).value

    # Accuracy Summary
    acc_records = []
    for ps, rs in prs:
        sub_l2 = df_ds[(df_ds["nmf_method"]=="L2_norm_NMF") &
                       (df_ds["p"]==ps) & (df_ds["r"]==rs)]
        sub_l21 = df_ds[(df_ds["nmf_method"]=="L21_norm_based_NMF") &
                        (df_ds["p"]==ps) & (df_ds["r"]==rs)]
        acc_records.append([
            f"p={ps}, r={rs}",
            "L2",
            float(sub_l2["acc_mean"].values[0]),
            float(sub_l2["acc_std"].values[0]),
            "L21",
            float(sub_l21["acc_mean"].values[0]),
            float(sub_l21["acc_std"].values[0]),
        ])
    acc_summary = pd.DataFrame(acc_records, columns=[
        "pr", "L2", "L2_mean", "L2_std",
        "L21", "L21_mean", "L21_std"
    ])
    print(f"\nDataset {ds} Accuracy Summary")
    print(acc_summary)
    result_logger.info(f"\nDataset {ds} ACC Summary")
    result_logger.info(f'{acc_summary}')

    # NMI Summary
    nmi_records = []
    for ps, rs in prs:
        sub_l2 = df_ds[(df_ds["nmf_method"]=="L2_norm_NMF") &
                       (df_ds["p"]==ps) & (df_ds["r"]==rs)]
        sub_l21 = df_ds[(df_ds["nmf_method"]=="L21_norm_based_NMF") &
                        (df_ds["p"]==ps) & (df_ds["r"]==rs)]
        nmi_records.append([
            f"p={ps}, r={rs}",
            "L2",
            float(sub_l2["nmi_mean"].values[0]),
            float(sub_l2["nmi_std"].values[0]),
            "L21",
            float(sub_l21["nmi_mean"].values[0]),
            float(sub_l21["nmi_std"].values[0]),
        ])
    nmi_summary = pd.DataFrame(nmi_records, columns=[
        "pr", "L2", "L2_mean", "L2_std",
        "L21", "L21_mean", "L21_std"
    ])
    print(f"\nDataset {ds} NMI Summary")
    print(nmi_summary)
    result_logger.info(f"\nDataset {ds} NMI Summary")
    result_logger.info(f'{nmi_summary}')
```

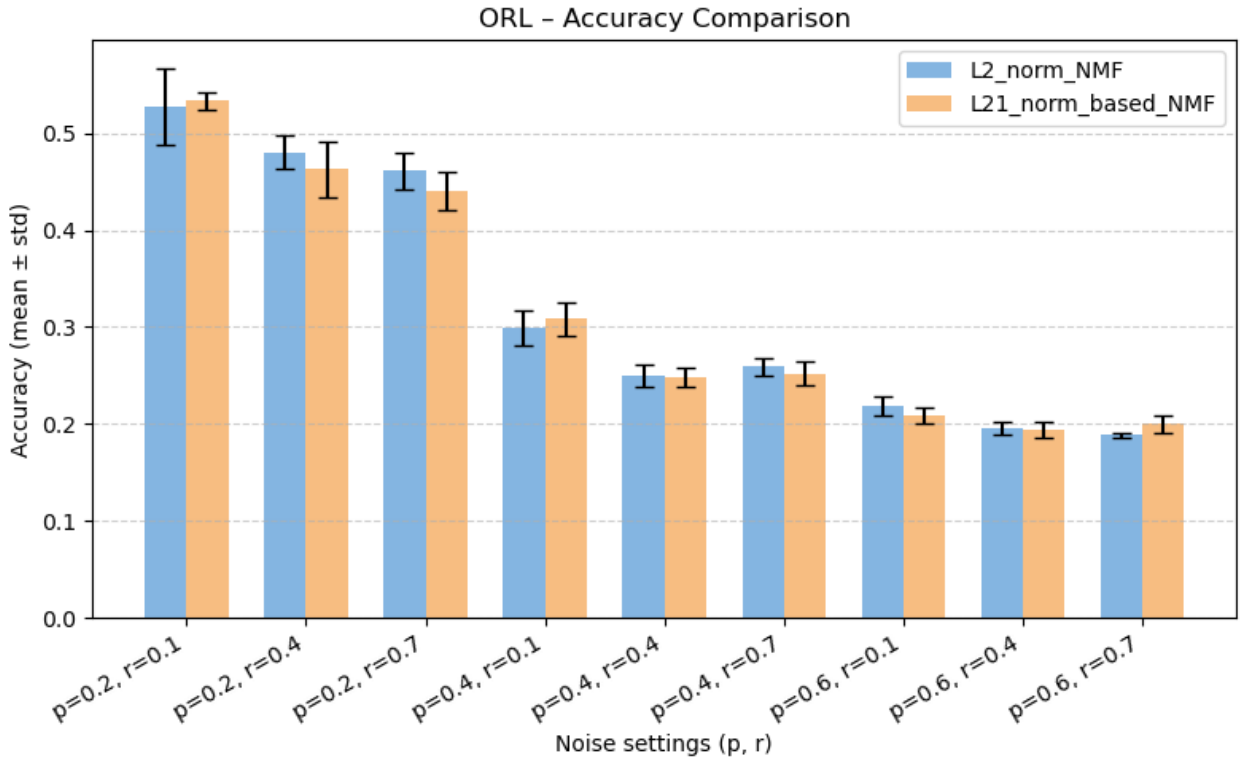
```
Dataset ORL Accuracy Summary
             pr  L2   L2_mean    L2_std  L21  L21_mean   L21_std
0  p=0.2, r=0.1  L2  0.527778  0.039830  L21  0.533889  0.009196
1  p=0.2, r=0.4  L2  0.480556  0.017830  L21  0.462778  0.028577
2  p=0.2, r=0.7  L2  0.461667  0.018873  L21  0.440556  0.019452
3  p=0.4, r=0.1  L2  0.299444  0.018291  L21  0.308333  0.017392
4  p=0.4, r=0.4  L2  0.249444  0.011574  L21  0.248333  0.009230
5  p=0.4, r=0.7  L2  0.259444  0.009230  L21  0.252222  0.012595
6  p=0.6, r=0.1  L2  0.218889  0.009526  L21  0.208889  0.008678
7  p=0.6, r=0.4  L2  0.195556  0.006236  L21  0.193333  0.008352
8  p=0.6, r=0.7  L2  0.188333  0.002079  L21  0.200000  0.008784

Dataset ORL NMI Summary
             pr  L2   L2_mean    L2_std  L21  L21_mean   L21_std
0  p=0.2, r=0.1  L2  0.690813  0.030305  L21  0.694755  0.007602
1  p=0.2, r=0.4  L2  0.643507  0.010367  L21  0.638062  0.028897
2  p=0.2, r=0.7  L2  0.629385  0.014422  L21  0.620242  0.013712
3  p=0.4, r=0.1  L2  0.492617  0.015867  L21  0.497793  0.012006
4  p=0.4, r=0.4  L2  0.436613  0.017451  L21  0.437632  0.014093
5  p=0.4, r=0.7  L2  0.441624  0.015249  L21  0.431450  0.020534
6  p=0.6, r=0.1  L2  0.398566  0.009481  L21  0.366084  0.012294
7  p=0.6, r=0.4  L2  0.383614  0.008029  L21  0.369256  0.007453
8  p=0.6, r=0.7  L2  0.371728  0.012863  L21  0.376916  0.016814

Dataset YaleB Accuracy Summary
             pr  L2   L2_mean    L2_std  L21  L21_mean   L21_std
0  p=0.2, r=0.1  L2  0.224217  0.011917  L21  0.206538  0.014465
1  p=0.2, r=0.4  L2  0.207643  0.011157  L21  0.207735  0.006546
2  p=0.2, r=0.7  L2  0.197145  0.010677  L21  0.214917  0.005341
3  p=0.4, r=0.1  L2  0.174770  0.012015  L21  0.174954  0.008394
4  p=0.4, r=0.4  L2  0.147422  0.004487  L21  0.149448  0.003034
5  p=0.4, r=0.7  L2  0.139411  0.009057  L21  0.142541  0.001663
6  p=0.6, r=0.1  L2  0.099171  0.004412  L21  0.103591  0.003345
7  p=0.6, r=0.4  L2  0.090424  0.002582  L21  0.090700  0.003396
8  p=0.6, r=0.7  L2  0.090331  0.001983  L21  0.090055  0.003048

Dataset YaleB NMI Summary
             pr  L2   L2_mean    L2_std  L21  L21_mean   L21_std
0  p=0.2, r=0.1  L2  0.307366  0.012012  L21  0.291029  0.012011
1  p=0.2, r=0.4  L2  0.288455  0.009477  L21  0.282401  0.006273
2  p=0.2, r=0.7  L2  0.284925  0.009236  L21  0.290274  0.013864
3  p=0.4, r=0.1  L2  0.230837  0.013544  L21  0.228220  0.005269
4  p=0.4, r=0.4  L2  0.187951  0.008512  L21  0.190000  0.006190
5  p=0.4, r=0.7  L2  0.172081  0.009829  L21  0.171786  0.012068
6  p=0.6, r=0.1  L2  0.109132  0.006811  L21  0.116367  0.007354
7  p=0.6, r=0.4  L2  0.092143  0.006386  L21  0.090926  0.002713
8  p=0.6, r=0.7  L2  0.093060  0.003135  L21  0.090465  0.003612
```

In [31]:

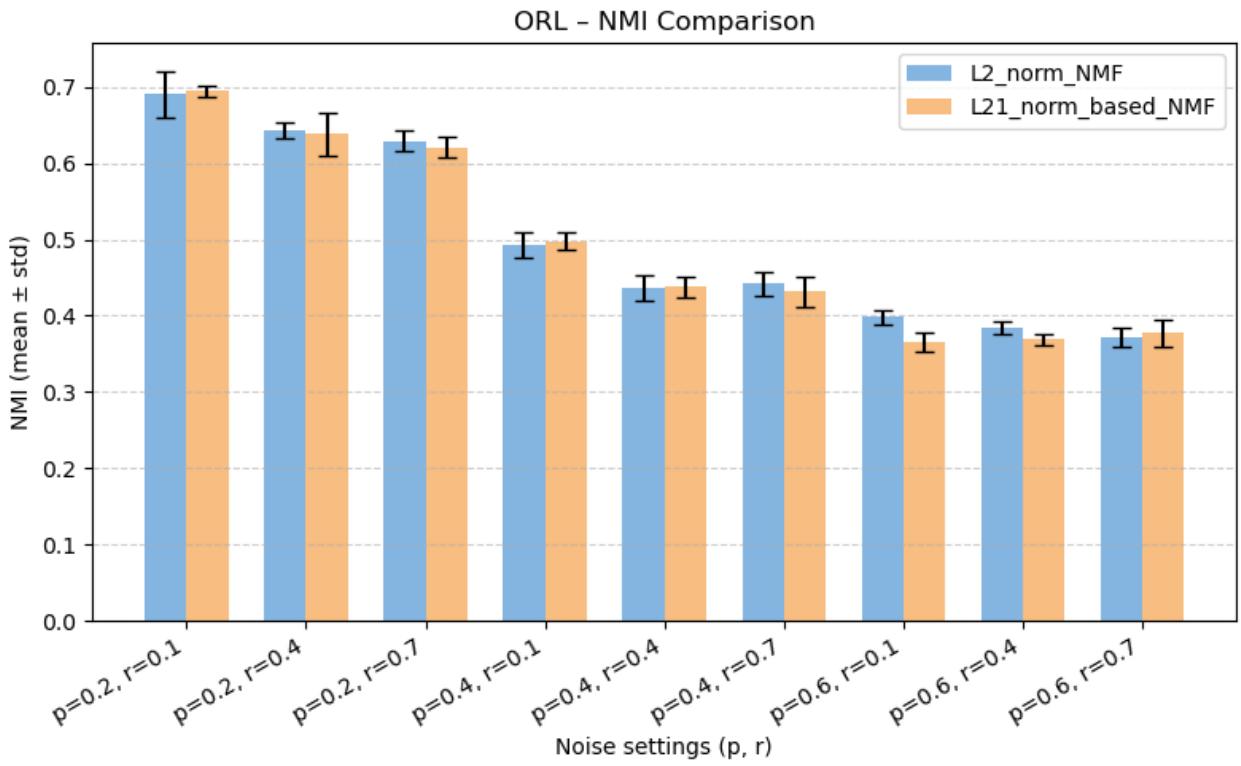
```python
datasets = ['ORL', 'YaleB']
methods = ['L2_norm_NMF', 'L21_norm_based_NMF']
metrics = [
    ('acc_mean', 'acc_std', 'Accuracy'),
    ('nmi_mean', 'nmi_std', 'NMI')
]
colors = ['#6fa8dc', '#f6b26b']

for dataset in datasets:
```

```
    sub_df = df_acc_nmi[df_acc_nmi['dataset'] == dataset]

    for mean_col, std_col, metric_name in metrics:
        sub_df['p_r'] = sub_df.apply(lambda row: f"p={row['p']}, r={row['
        x_labels = sub_df['p_r'].unique()
        x = np.arange(len(x_labels))
        width = 0.35

        fig, ax = plt.subplots(figsize=(8,5))
        for i, method in enumerate(methods):
            temp = sub_df[sub_df['nmf_method'] == method]
            means = temp[mean_col].values
            stds = temp[std_col].values

            ax.bar(
                x + (i - 0.5) * width,
                means,
                width,
                yerr=stds,
                capsize=4,
                label=method,
                color=colors[i],
                alpha=0.85
            )

        ax.set_xticks(x)
        ax.set_xticklabels(x_labels, rotation=30, ha='right')
        ax.set_xlabel('Noise settings (p, r)')
        ax.set_ylabel(f'{metric_name} (mean ± std)')
        ax.set_title(f'{dataset} - {metric_name} Comparison')
        ax.legend()
        ax.grid(axis='y', linestyle='--', alpha=0.6)
        plt.tight_layout()
        plt.show()
```

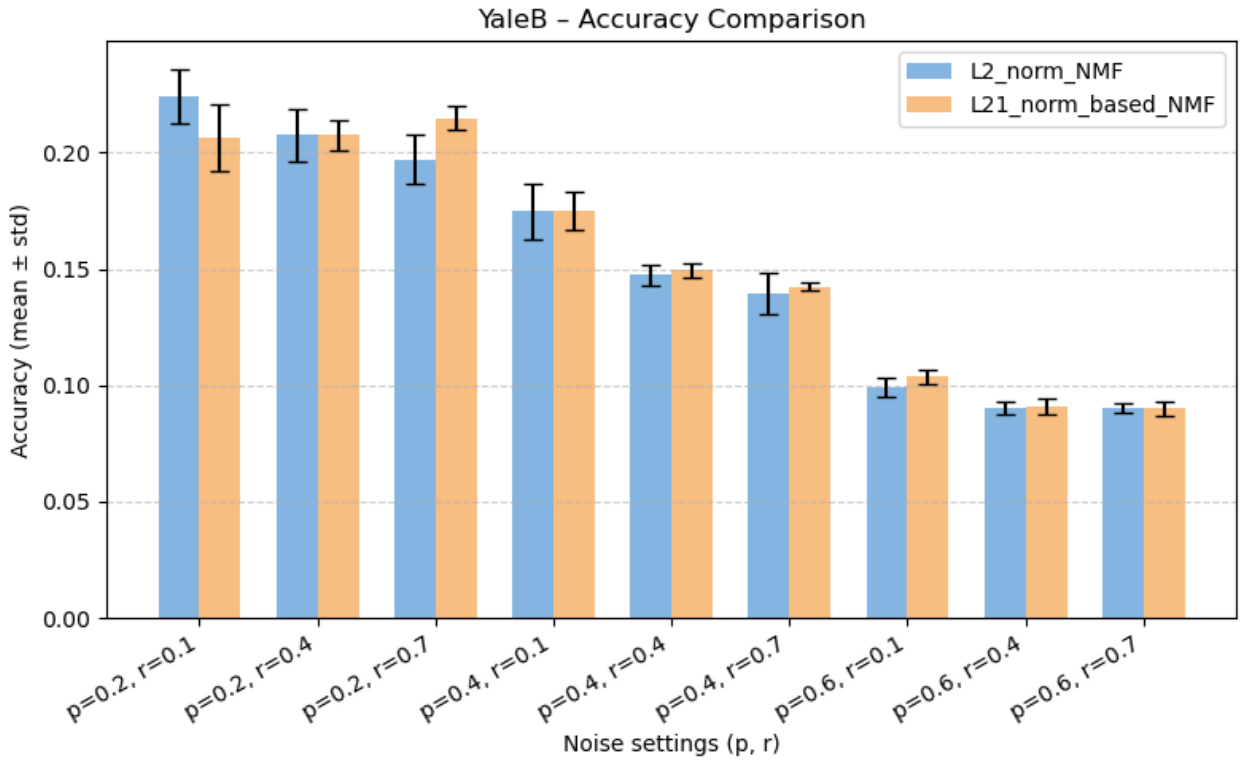
```
/var/folders/yf/j_qldhjn37d5zwpg9dwkd6t80000gn/T/ipykernel_35088/102215509
4.py:13: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-doc
s/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
  sub_df['p_r'] = sub_df.apply(lambda row: f"p={row['p']}, r={row['r']}",
axis=1)
```



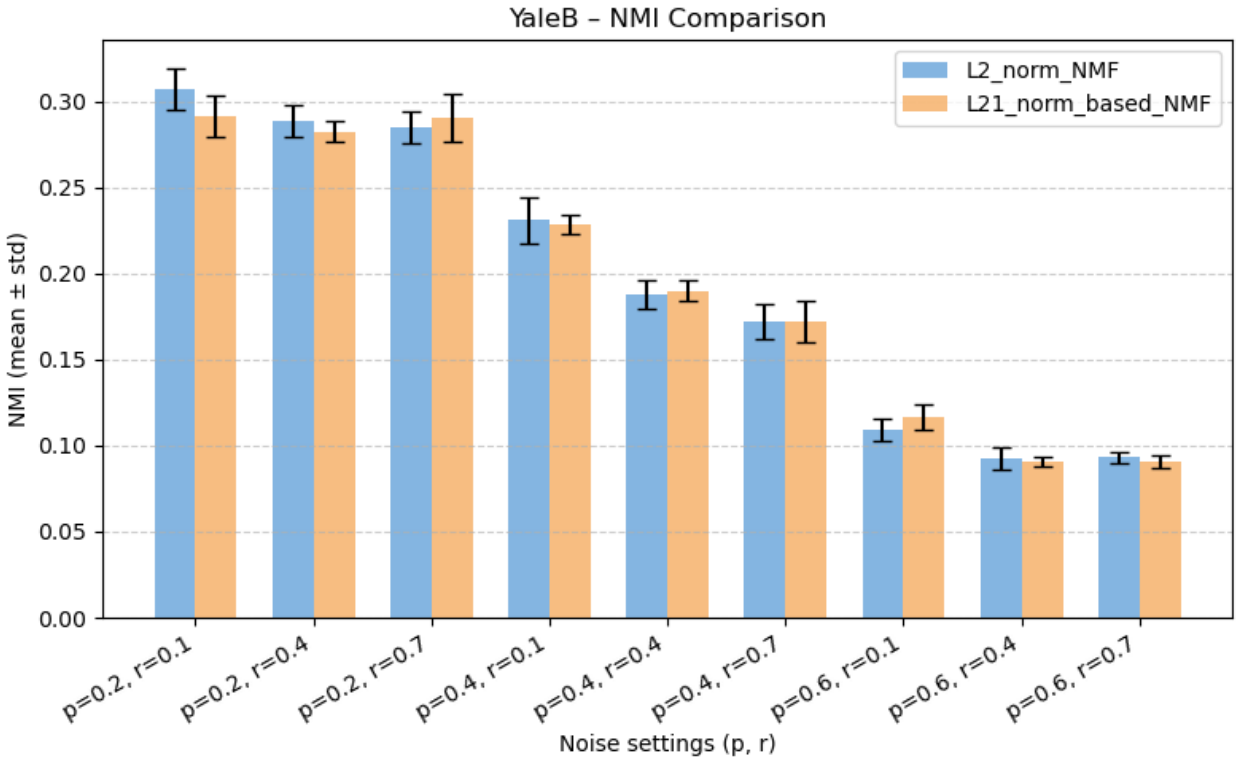

```
/var/folders/yf/j_qldhjn37d5zwpg9dwkd6t80000gn/T/ipykernel_35088/102215509
4.py:13: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-doc
s/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
  sub_df['p_r'] = sub_df.apply(lambda row: f"p={row['p']}, r={row['r']}",
axis=1)
```



```
/var/folders/yf/j_qldhjn37d5zwpg9dwkd6t80000gn/T/ipykernel_35088/102215509
4.py:13: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-doc
s/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
  sub_df['p_r'] = sub_df.apply(lambda row: f"p={row['p']}, r={row['r']}",
axis=1)
```




```
/var/folders/yf/j_qldhjn37d5zwpg9dwkd6t80000gn/T/ipykernel_35088/102215509
4.py:13: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-doc
s/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
  sub_df['p_r'] = sub_df.apply(lambda row: f"p={row['p']}, r={row['r']}",
axis=1)
```



In [32]:
```
# Table Visualization
metrics_2 = ["acc_mean","acc_std","nmi_mean","nmi_std"]
print('The value of columns is r while the value of index is p.')
for metric, name, method in product(metrics_2, datasets, methods):
    print('-')
    print(f'{metric} for dataset "{name}" using algorithm "{method}":')
    result_logger.info(f'{metric} for dataset "{name}" using algorithm "{
    sub = df_acc_nmi[(df_acc_nmi["dataset"] == name) & (df_acc_nmi["nmf_m
    table_visual(sub, metric)
```

```
The value of columns is r while the value of index is p.
-
acc_mean for dataset "ORL" using algorithm "L2_norm_NMF":
r         0.1       0.4       0.7
p
0.2  0.527778  0.480556  0.461667
0.4  0.299444  0.249444  0.259444
0.6  0.218889  0.195556  0.188333
-
acc_mean for dataset "ORL" using algorithm "L21_norm_based_NMF":
r         0.1       0.4       0.7
p
0.2  0.533889  0.462778  0.440556
0.4  0.308333  0.248333  0.252222
0.6  0.208889  0.193333  0.200000
-
acc_mean for dataset "YaleB" using algorithm "L2_norm_NMF":
r         0.1       0.4       0.7
p
0.2  0.224217  0.207643  0.197145
0.4  0.174770  0.147422  0.139411
0.6  0.099171  0.090424  0.090331
-
acc_mean for dataset "YaleB" using algorithm "L21_norm_based_NMF":
r         0.1       0.4       0.7
```

```
p
0.2  0.206538  0.207735  0.214917
0.4  0.174954  0.149448  0.142541
0.6  0.103591  0.090700  0.090055
-
acc_std for dataset "ORL" using algorithm "L2_norm_NMF":
r         0.1       0.4       0.7
p
0.2  0.039830  0.017830  0.018873
0.4  0.018291  0.011574  0.009230
0.6  0.009526  0.006236  0.002079
-
acc_std for dataset "ORL" using algorithm "L21_norm_based_NMF":
r         0.1       0.4       0.7
p
0.2  0.009196  0.028577  0.019452
0.4  0.017392  0.009230  0.012595
0.6  0.008678  0.008352  0.008784
-
acc_std for dataset "YaleB" using algorithm "L2_norm_NMF":
r         0.1       0.4       0.7
p
0.2  0.011917  0.011157  0.010677
0.4  0.012015  0.004487  0.009057
0.6  0.004412  0.002582  0.001983
-
acc_std for dataset "YaleB" using algorithm "L21_norm_based_NMF":
r         0.1       0.4       0.7
p
0.2  0.014465  0.006546  0.005341
0.4  0.008394  0.003034  0.001663
0.6  0.003345  0.003396  0.003048
-
nmi_mean for dataset "ORL" using algorithm "L2_norm_NMF":
r         0.1       0.4       0.7
p
0.2  0.690813  0.643507  0.629385
0.4  0.492617  0.436613  0.441624
0.6  0.398566  0.383614  0.371728
-
nmi_mean for dataset "ORL" using algorithm "L21_norm_based_NMF":
r         0.1       0.4       0.7
p
0.2  0.694755  0.638062  0.620242
0.4  0.497793  0.437632  0.431450
0.6  0.366084  0.369256  0.376916
-
nmi_mean for dataset "YaleB" using algorithm "L2_norm_NMF":
r         0.1       0.4       0.7
p
0.2  0.307366  0.288455  0.284925
0.4  0.230837  0.187951  0.172081
0.6  0.109132  0.092143  0.093060
-
nmi_mean for dataset "YaleB" using algorithm "L21_norm_based_NMF":
r         0.1       0.4       0.7
p
```

```
0.2  0.291029  0.282401  0.290274
0.4  0.228220  0.190000  0.171786
0.6  0.116367  0.090926  0.090465
-
nmi_std for dataset "ORL" using algorithm "L2_norm_NMF":
r         0.1       0.4       0.7
p
0.2  0.030305  0.010367  0.014422
0.4  0.015867  0.017451  0.015249
0.6  0.009481  0.008029  0.012863
-
nmi_std for dataset "ORL" using algorithm "L21_norm_based_NMF":
r         0.1       0.4       0.7
p
0.2  0.007602  0.028897  0.013712
0.4  0.012006  0.014093  0.020534
0.6  0.012294  0.007453  0.016814
-
nmi_std for dataset "YaleB" using algorithm "L2_norm_NMF":
r         0.1       0.4       0.7
p
0.2  0.012012  0.009477  0.009236
0.4  0.013544  0.008512  0.009829
0.6  0.006811  0.006386  0.003135
-
nmi_std for dataset "YaleB" using algorithm "L21_norm_based_NMF":
r         0.1       0.4       0.7
p
0.2  0.012011  0.006273  0.013864
0.4  0.005269  0.006190  0.012068
0.6  0.007354  0.002713  0.003612
```